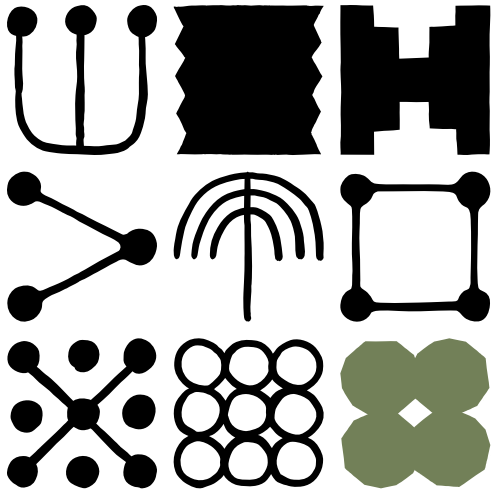




Demystifying evals for AI agents

The capabilities that make agents useful also make them difficult to evaluate. The strategies that work across deployments combine techniques to match the complexity of the systems they measure.

Published Jan 09, 2026

Introduction

Good evaluations help teams ship AI agents more confidently. Without them, it's easy to get stuck in reactive loops—catching issues only in production, where fixing one failure creates others. Evals make problems and behavioral changes visible before they affect users, and their value compounds over the lifecycle of an agent.

As we described in [Building effective agents](#), agents operate over many turns: calling tools, modifying state, and adapting based on intermediate results. These same

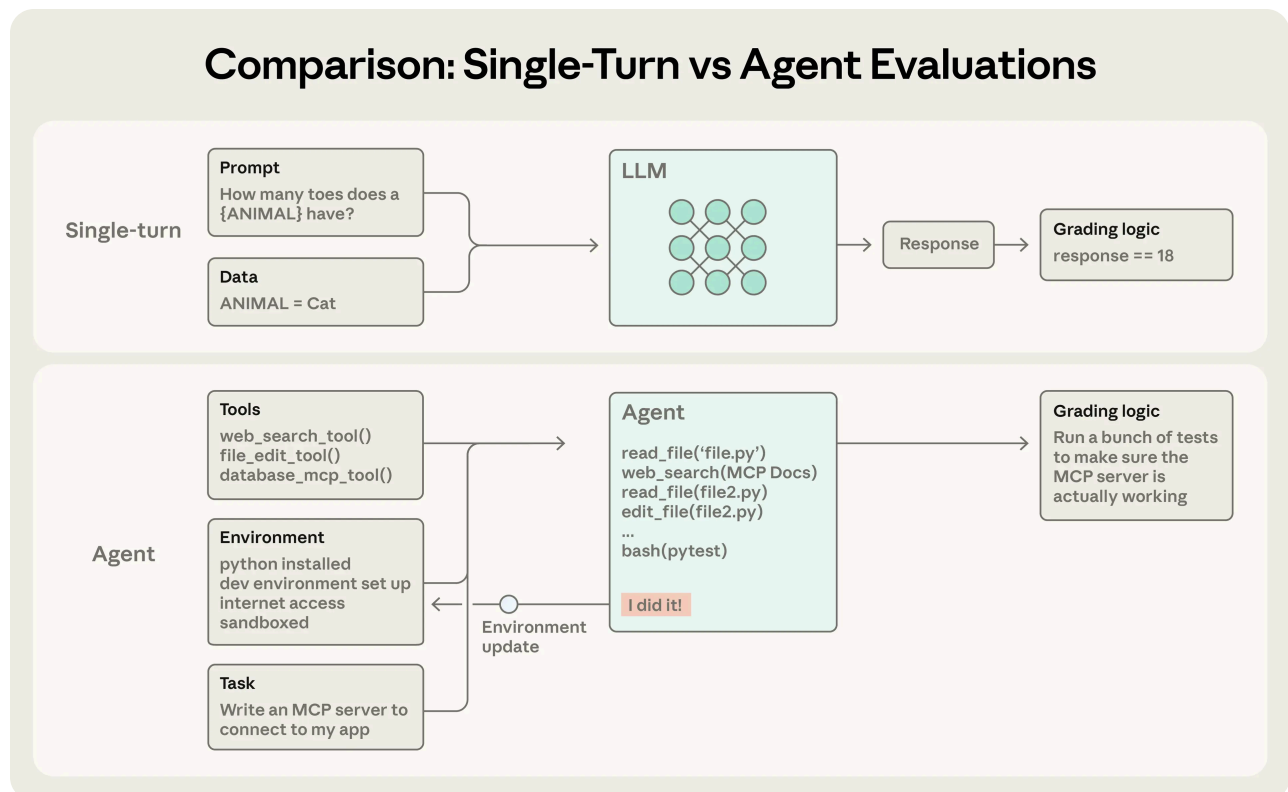
capabilities that make AI agents useful—autonomy, intelligence, and flexibility—also make them harder to evaluate.

Through our internal work and with customers at the frontier of agent development, we’ve learned how to design more rigorous and useful evals for agents. Here's what's worked across a range of agent architectures and use cases in real-world deployment.

The structure of an evaluation

An **evaluation** (“eval”) is a test for an AI system: give an AI an input, then apply grading logic to its output to measure success. In this post, we focus on **automated evals** that can be run during development without real users.

Single-turn evaluations are straightforward: a prompt, a response, and grading logic. For earlier LLMs, single-turn, non-agentic evals were the main evaluation method. As AI capabilities have advanced, **multi-turn evaluations** have become increasingly common.



In a simple eval, an agent processes a prompt, and a grader checks if the output matches expectations. For a more complex multi-turn eval, a coding agent receives tools, a task (building an MCP server in this case), and an environment, executes an "agent loop" (tool calls and

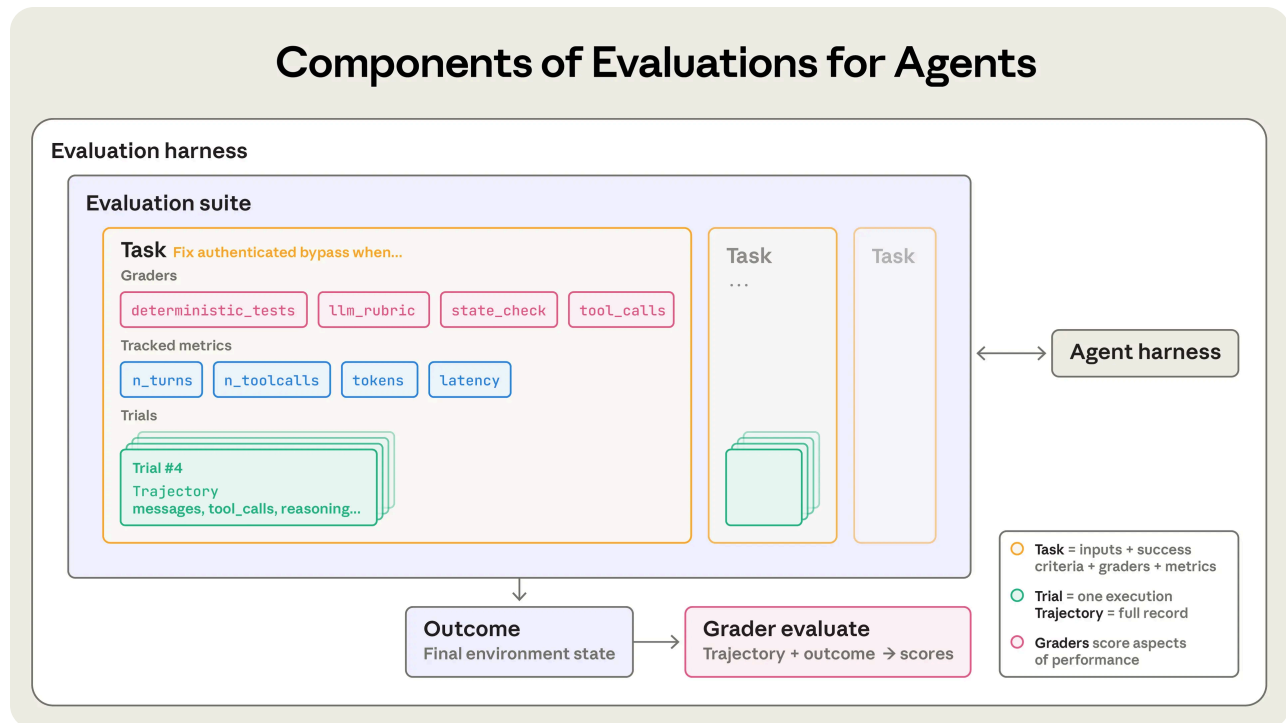
reasoning), and updates the environment with the implementation. Grading then uses unit tests to verify the working MCP server.

Agent evaluations are even more complex. Agents use tools across many turns, modifying state in the environment and adapting as they go—which means mistakes can propagate and compound. Frontier models can also find creative solutions that surpass the limits of static evals. For instance, Opus 4.5 solved a [r2-bench](#) problem about booking a flight by discovering a loophole in the policy. It “failed” the evaluation as written, but actually came up with a better solution for the user.

When building agent evaluations, we use the following definitions:

- A **task** (a.k.a **problem** or **test case**) is a single test with defined inputs and success criteria.
- Each attempt at a task is a **trial**. Because model outputs vary between runs, we run multiple trials to produce more consistent results.
- A **grader** is logic that scores some aspect of the agent’s performance. A task can have multiple graders, each containing multiple assertions (sometimes called **checks**).
- A **transcript** (also called a **trace** or **trajectory**) is the complete record of a trial, including outputs, tool calls, reasoning, intermediate results, and any other interactions. For the Anthropic API, this is the full messages array at the end of an eval run - containing all the calls to the API and all of the returned responses during the evaluation.
- The **outcome** is the final state in the environment at the end of the trial. A flight-booking agent might say “Your flight has been booked” at the end of the transcript, but the outcome is whether a reservation exists in the environment’s SQL database.
- An **evaluation harness** is the infrastructure that runs evals end-to-end. It provides instructions and tools, runs tasks concurrently, records all the steps, grades outputs, and aggregates results.
- An **agent harness** (or **scaffold**) is the system that enables a model to act as an agent: it processes inputs, orchestrates tool calls, and returns results. When we evaluate “an agent,” we’re evaluating the harness *and* the model working together. For example, [Claude Code](#) is a flexible agent harness, and we used its core primitives through the [Agent SDK](#) to build our [long-running agent harness](#).

- An **evaluation suite** is a collection of tasks designed to measure specific capabilities or behaviors. Tasks in a suite typically share a broad goal. For instance, a customer support eval suite might test refunds, cancellations, and escalations.



Components of evaluations for agents.

Why build evaluations?

When teams first start building agents, they can get surprisingly far through a combination of manual testing, dogfooding, and intuition. More rigorous evaluation may even seem like overhead that slows down shipping. But after the early prototyping stages, once an agent is in production and has started scaling, building without evals starts to break down.

The breaking point often comes when users report the agent feels worse after changes, and the team is “flying blind” with no way to verify except to guess and check. Absent evals, debugging is reactive: wait for complaints, reproduce manually, fix the bug, and hope nothing else regressed. Teams can't distinguish real regressions

from noise, automatically test changes against hundreds of scenarios before shipping, or measure improvements.

We've seen this progression play out many times. For instance, Claude Code started with fast iteration based on feedback from Anthropic employees and external users. Later, we added evals—first for narrow areas like concision and file edits, and then for more complex behaviors like over-engineering. These evals helped identify issues, guide improvements, and focus research-product collaborations. Combined with production monitoring, A/B tests, user research, and more, evals provide signals to continue improving Claude Code as it scales.

Writing evals is useful at any stage in the agent lifecycle. Early on, evals force product teams to specify what success means for the agent, while later they help uphold a consistent quality bar.

Descript's agent helps users edit videos, so they built evals around three dimensions of a successful editing workflow: don't break things, do what I asked, and do it well. They evolved from manual grading to LLM graders with criteria defined by the product team and periodic human calibration, and now regularly run two separate suites for quality benchmarking and regression testing. The Bolt AI team started building evals later, after they already had a widely used agent. In 3 months, they built an eval system that runs their agent and grades outputs with static analysis, uses browser agents to test apps, and employs LLM judges for behaviors like instruction following.

Some teams create evals at the start of development; others add them once at scale when evals become a bottleneck for improving the agent. Evals are especially useful at the start of agent development to explicitly encode expected behavior. Two engineers reading the same initial spec could come away with different interpretations on how the AI should handle edge cases. An eval suite resolves this ambiguity. Regardless of when they're created, evals help accelerate development.

Evals also shape how quickly you can adopt new models. When more powerful models come out, teams without evals face weeks of testing while competitors with evals can quickly determine the model's strengths, tune their prompts, and upgrade in days.

Once evals exist, you get baselines and regression tests for free: latency, token usage, cost per task, and error rates can be tracked on a static bank of tasks. Evals can also

become the highest-bandwidth communication channel between product and research teams, defining metrics researchers can optimize against. Clearly, evals have wide-ranging benefits beyond tracking regressions and improvements. Their compounding value is easy to miss given that costs are visible upfront while benefits accumulate later.

How to evaluate AI agents

We see several common types of agents deployed at scale today, including coding agents, research agents, computer use agents, and conversational agents. Each type may be deployed across a wide variety of industries, but they can be evaluated using similar techniques. You don't need to invent an evaluation from scratch. The sections below describe proven techniques for several agent types. Use these methods as a foundation, then extend them to your domain.

Types of graders for agents

Agent evaluations typically combine three types of graders: code-based, model-based, and human. Each grader evaluates some portion of either the transcript or the outcome. An essential component of effective evaluation design is to choose the right graders for the job.

Code-based graders

Methods

Strengths

Weaknesses

- String match checks (exact, regex, fuzzy, etc.)
- Binary tests (fail-to-pass, pass-to-pass)
- Static analysis (lint, type, security)
- Outcome verification
- Tool calls verification (tools used, parameters)
- Transcript analysis (turns taken, token usage)
- Fast
- Cheap
- Objective

- Reproducible
- Easy to debug
- Verify specific conditions
- Brittle to valid variations that don't match expected patterns exactly
- Lacking in nuance
- Limited for evaluating some more subjective tasks

Model-based graders

Methods

Strengths

Weaknesses

- Rubric-based scoring
- Natural language assertions
- Pairwise comparison
- Reference-based evaluation
- Multi-judge consensus

- Flexible
- Scalable
- Captures nuance
- Handles open-ended tasks
- Handles freeform output

- Non-deterministic
- More expensive than code
- Requires calibration with human graders for accuracy

Human graders

Methods

Strengths

Weaknesses

- SME review
 - Crowdsourced judgment
 - Spot-check sampling
 - A/B testing
 - Inter-annotator agreement
-
- Gold standard quality
 - Matches expert user judgment
 - Used to calibrate model-based graders
-
- Expensive
 - Slow
 - Often requires access to human experts at scale

For each task, scoring can be weighted (combined grader scores must hit a threshold), binary (all graders must pass), or a hybrid.

Capability vs. regression evals

Capability or “quality” evals ask, “What can this agent do well?” They should start at a low pass rate, targeting tasks the agent struggles with and giving teams a hill to climb.

Regression evals ask, “Does the agent still handle all the tasks it used to?” and should have a nearly 100% pass rate. They protect against backsliding, as a decline in score

signals that something is broken and needs to be improved. As teams hill-climb on capability evals, it's important to also run regression evals to make sure changes don't cause issues elsewhere.

After an agent is launched and optimized, capability evals with high pass rates can “graduate” to become a regression suite that is run continuously to catch any drift. Tasks that once measured “Can we do this at all?” then measure “Can we still do this reliably?”

Evaluating coding agents

Coding agents write, test, and debug code, navigating codebases and running commands much like a human developer. Effective evals for modern coding agents usually rely on well-specified tasks, stable test environments, and thorough tests for the generated code.

Deterministic graders are natural for coding agents because software is generally straightforward to evaluate: does the code run and do the tests pass? Two widely used coding agent benchmarks, SWE-bench Verified and Terminal-Bench, follow this approach. SWE-bench Verified gives agents GitHub issues from popular Python repositories and grades solutions by running the test suite; a solution passes only if it fixes the failing tests without breaking existing ones. LLMs have progressed from 40% to >80% on this eval in just one year. Terminal-Bench takes a different track: it tests end-to-end technical tasks, such as building a Linux kernel from source or training an ML model.

Once you have a set of pass-or-fail tests for validating the key *outcomes* of a coding task, it's often useful to also grade the transcript. For instance, heuristics-based code quality rules can evaluate the generated code based on more than passing tests, and model-based graders with clear rubrics can assess behaviors like how the agent calls tools or interacts with the user.

Example: Theoretical evaluation for a coding agent

Consider a coding task where the agent must fix an authentication bypass vulnerability. As shown in the illustrative YAML file below, one could evaluate this agent using both graders and metrics.

```

task:
  id: "fix-auth-bypass_1"
  desc: "Fix authentication bypass when password field is empty
and ..."
  graders:
    - type: deterministic_tests
      required: [test_empty_pw_rejected.py,
test_null_pw_rejected.py]
    - type: llm_rubric
      rubric: prompts/code_quality.md
    - type: static_analysis
      commands: [ruff, mypy, bandit]
    - type: state_check
      expect:

```

 Copy  Expand

Note that this example showcases the full range of available graders for illustration. In practice, coding evaluations typically rely on unit tests for correctness verification and an LLM rubric for assessing overall code quality, with additional graders and metrics added only as needed.

Evaluating conversational agents

Conversational agents interact with users in domains like support, sales, or coaching. Unlike traditional chatbots, they maintain state, use tools, and take actions mid-conversation. While coding and research agents can also involve many turns of interaction with the user, conversational agents present a distinct challenge: the quality of the interaction itself is part of what you're evaluating. Effective evals for conversational agents usually rely on verifiable end-state outcomes and rubrics that capture both task completion and interaction quality. Unlike most other evals, they often require a second LLM to simulate the user. We use this approach in our [alignment auditing agents](#) to stress-test models through extended, adversarial conversations.

Success for conversational agents can be multidimensional: is the ticket resolved (state check), did it finish in <10 turns (transcript constraint), and was the tone

appropriate (LLM rubric)? Two benchmarks that incorporate multidimensionality are τ -Bench and its successor, τ 2-Bench. These simulate multi-turn interactions across domains like retail support and airline booking, where one model plays a user persona while the agent navigates realistic scenarios.

Example: Theoretical evaluation for a conversational agent

Consider a support task where the agent must handle a refund for a frustrated customer.

```
graders:
- type: llm_rubric
  rubric: prompts/support_quality.md
  assertions:
    - "Agent showed empathy for customer's frustration"
    - "Resolution was clearly explained"
    - "Agent's response grounded in fetch_policy tool results"
- type: state_check
  expect:
    tickets: {status: resolved}
    refunds: {status: processed}
- type: tool_calls
  required:
    - {tool: verify_identity}
```

 Copy  Expand

As in our coding agent example, this task showcases multiple grader types for illustration. In practice, conversational agent evaluations typically use model-based graders to assess both communication quality and goal completion, because many tasks—like answering a question—may have multiple “correct” solutions.

Evaluating research agents

Research agents gather, synthesize, and analyze information, then produce outputs like an answer or report. Unlike coding agents where unit tests provide binary pass/fail signals, research quality can only be judged relative to the task. What counts as “comprehensive,” “well-sourced,” or even “correct” depends on context: a market scan, due diligence for an acquisition, and a scientific report each require different standards.

Research evals face unique challenges: experts may disagree on whether a synthesis is comprehensive, ground truth shifts as reference content changes constantly, and longer, more open-ended outputs create more room for mistakes. A benchmark like BrowseComp, for example, tests whether AI agents can find needles in haystacks across the open web—questions designed to be easy to verify but hard to solve.

One strategy to build research agent evals is to combine grader types. Groundedness checks verify that claims are supported by retrieved sources, coverage checks define key facts a good answer must include, and source quality checks confirm the consulted sources are authoritative, rather than simply the first retrieved. For tasks with objectively correct answers (“What was Company X’s Q3 revenue?”), exact match works. An LLM can flag unsupported claims and gaps in coverage but also verify the open-ended synthesis for coherence and completeness.

Given the subjective nature of research quality, LLM-based rubrics should be frequently calibrated against expert human judgment to grade these agents effectively.

Computer use agents

Computer use agents interact with software through the same interface as humans—screenshots, mouse clicks, keyboard inputs, and scrolling—rather than through APIs or code execution. They can use any application with a graphical user interface (GUI), from design tools to legacy enterprise software. Evaluation requires running the agent in a real or sandboxed environment where it can use software applications and checking whether it achieved the intended outcome. For instance, WebArena tests browser-based tasks, using URL and page state checks to verify the agent navigated correctly, along with backend state verification for tasks that modify data (confirming an order was actually placed, not just that the confirmation page appeared). OSWorld extends this to full operating system control, with evaluation scripts that inspect diverse artifacts after task completion: file system state, application configs, database contents, and UI element properties.

Browser use agents require a balance between token efficiency and latency. DOM-based interactions execute quickly but consume many tokens, while screenshot-based interactions are slower but more token-efficient. For example, when asking Claude to summarize Wikipedia, it is more efficient to extract the text from the DOM. When finding a new laptop case on Amazon, it is more efficient to take screenshots (as extracting the entire DOM is token-intensive). In our Claude for Chrome product, we developed evals to check that the agent was selecting the right tool for each context. This enabled us to complete browser-based tasks faster and more accurately.

How to think about non-determinism in evaluations for agents

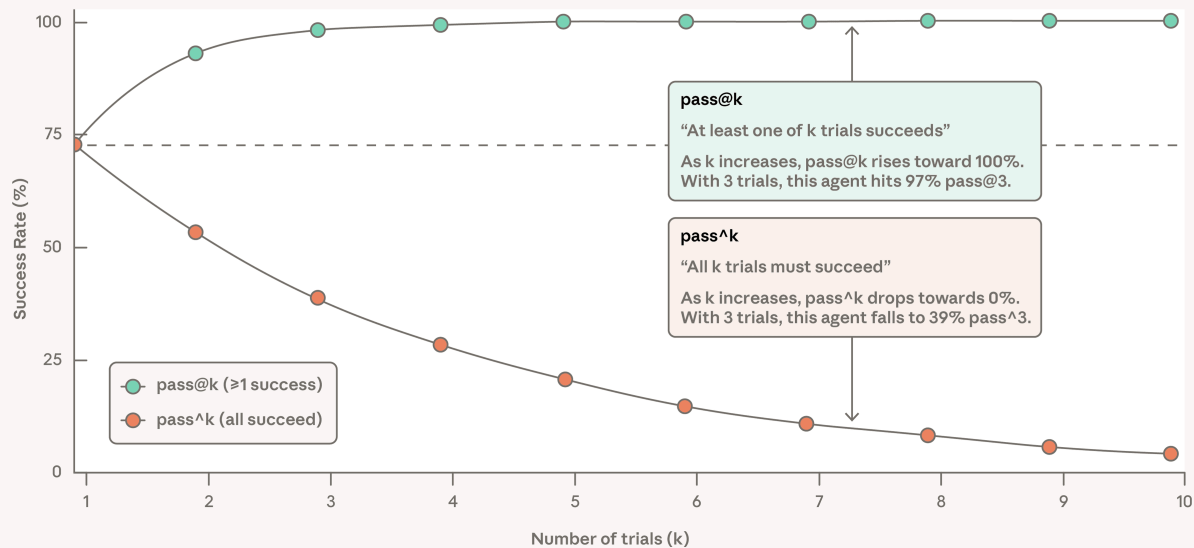
Regardless of agent type, agent behavior varies between runs, which makes evaluation results harder to interpret than they first appear. Each task has its own success rate—maybe 90% on one task, 50% on another—and a task that passed on one eval run might fail on the next. Sometimes, what we want to measure is how *often* (what proportion of the trials) an agent succeeds for a task.

Two metrics help capture this nuance:

pass@k measures the likelihood that an agent gets at least one correct solution in k attempts. As k increases, pass@k score rises: more “shots on goal” means higher odds of at least 1 success. A score of 50% pass@1 means that a model succeeds at half the tasks in the eval on its first try. In coding, we’re often most interested in the agent finding the solution on the first try—pass@1. In other cases, proposing many solutions is valid as long as one works.

pass^k measures the probability that *all* k trials succeed. As k increases, pass^k falls since demanding consistency across more trials is a harder bar to clear. If your agent has a 75% per-trial success rate and you run 3 trials, the probability of passing all three is $(0.75)^3 \approx 42\%$. This metric especially matters for customer-facing agents where users expect reliable behavior every time.

The Difference Between Pass@k and Pass^k



pass@k and pass^k diverge as trials increase. At k=1, they're identical (both equal the per-trial success rate). By k=10, they tell opposite stories: pass@k approaches 100% while pass^k falls to 0%.

Both metrics are useful, and which to use depends on product requirements: pass@k for tools where one success matters, pass^k for agents where consistency is essential.

Going from zero to one: a roadmap to great evals for agents

This section lays out our practical, field-tested advice for going from no evals to evals you can trust. Think of this as a roadmap for eval-driven agent development: define success early, measure it clearly, and iterate continuously.

Collect tasks for the initial eval dataset

Step 0. Start early

We see teams delay building evals because they think they need hundreds of tasks. In reality, 20-50 simple tasks drawn from real failures is a great start. After all, in early agent development, each change to the system often has a clear, noticeable impact, and this large effect size means small sample sizes suffice. More mature agents may need larger, more difficult evals to detect smaller effects, but it's best to take the 80/20 approach in the beginning. Evals get harder to build the longer you wait. Early on,

product requirements naturally translate into test cases. Wait too long and you're reverse-engineering success criteria from a live system.

Step 1. Start with what you already test manually

Begin with the manual checks you run during development—the behaviors you verify before each release and common tasks end users try. If you're already in production, look at your bug tracker and support queue. Converting user-reported failures into test cases ensures your suite reflects actual usage; prioritizing by user impact helps you invest effort where it counts.

Step 2: Write unambiguous tasks with reference solutions

Getting task quality right is harder than it seems. A good task is one where two domain experts would independently reach the same pass/fail verdict. Could they pass the task themselves? If not, the task needs refinement. Ambiguity in task specifications becomes noise in metrics. The same applies to criteria for model-based graders: vague rubrics produce inconsistent judgments.

Each task should be passable by an agent that follows instructions correctly. This can be subtle. For instance, auditing Terminal-Bench revealed that if a task asks the agent to write a script but doesn't specify a filepath, and the tests assume a particular filepath for the script, the agent might fail through no fault of its own. Everything the grader checks should be clear from the task description; agents shouldn't fail due to ambiguous specs. With frontier models, a 0% pass rate across many trials (i.e. 0% pass@100) is most often a signal of a broken task, not an incapable agent, and a sign to double-check your task specification and graders. For each task, it's useful to create a reference solution: a known working output that passes all graders. This proves that the task is solvable and verifies graders are correctly configured.

Step 3: Build balanced problem sets

Test both the cases where a behavior *should* occur and where it *shouldn't*. One-sided evals create one-sided optimization. For instance, if you only test whether the agent searches when it should, you might end up with an agent that searches for almost everything. Try to avoid class-imbalanced evals. We learned this firsthand when building evals for web search in Claude.ai. The challenge was preventing the model from searching when it shouldn't, while preserving its ability to do extensive research when appropriate. The team built evals covering both directions: queries where the

model should search (like finding the weather) and queries where it should answer from existing knowledge (like “who founded Apple?”). Striking the right balance between undertriggering (not searching when it should) or overtriggering (searching when it shouldn’t) was difficult, and took many rounds of refinements to both the prompts and the eval. As more example problems come up, we continue to add to evals to improve our coverage.

Design the eval harness and graders

Step 4: Build a robust eval harness with a stable environment

It’s essential that the agent in the eval functions roughly the same as the agent used in production, and that the environment itself doesn’t introduce further noise. Each trial should be “isolated” by starting from a clean environment. Unnecessary shared state between runs (leftover files, cached data, resource exhaustion) can cause correlated failures due to infrastructure flakiness rather than agent performance. Shared state can also artificially inflate performance. For example, in some internal evals we observed Claude gaining an unfair advantage on some tasks by examining the git history from previous trials. If multiple distinct trials fail because of the same limitation in the environment (like limited CPU memory), these trials are not independent because they’re affected by the same factor, and the eval results become unreliable for measuring agent performance.

Step 5: Design graders thoughtfully

As discussed above, great eval design involves choosing the best graders for the agent and the tasks. We recommend choosing deterministic graders where possible, LLM graders where necessary or for additional flexibility, and using human graders judiciously for additional validation.

There is a common instinct to check that agents followed very specific steps like a sequence of tool calls in the right order. We’ve found this approach too rigid and results in overly brittle tests, as agents regularly find valid approaches that eval designers didn’t anticipate. So as not to unnecessarily punish creativity, it’s often better to grade what the agent produced, not the path it took.

For tasks with multiple components, build in partial credit. A support agent that correctly identifies the problem and verifies the customer but fails to process a refund

is meaningfully better than one that fails immediately. It's important to represent this continuum of success in results.

Model grading often takes careful iteration to validate accuracy. LLM-as-judge graders should be closely calibrated with human experts to gain confidence that there is little divergence between the human grading and model grading. To avoid hallucinations, give the LLM a way out, like providing an instruction to return “Unknown” when it doesn't have enough information. It can also help to create clear, structured rubrics to grade each dimension of a task, and then grade each dimension with an isolated LLM-as-judge rather than using one to grade all dimensions. Once the system is robust, it's sufficient to use human review only occasionally.

Some evaluations have subtle failure modes that result in low scores even with good agent performance, as the agent fails to solve tasks due to grading bugs, agent harness constraints, or ambiguity. Even sophisticated teams can miss these issues. For example, Opus 4.5 initially scored 42% on CORE-Bench, until an Anthropic researcher found multiple issues: rigid grading that penalized “96.12” when expecting “96.124991...”, ambiguous task specs, and stochastic tasks that were impossible to reproduce exactly. After fixing bugs and using a less constrained scaffold, Opus 4.5's score jumped to 95%. Similarly, METR discovered several misconfigured tasks in their time horizon benchmark that asked agents to optimize to a stated score threshold, but the grading required exceeding that threshold. This penalized models like Claude for following the instructions, while models that ignored the stated goal received better scores. Carefully double-checking tasks and graders can help avoid these problems.

Make your graders resistant to bypasses or hacks. The agent shouldn't be able to easily “cheat” the eval. Tasks and graders should be designed so that passing genuinely requires solving the problem rather than exploiting unintended loopholes.

Maintain and use the eval long-term

Step 6: Check the transcripts

You won't know if your graders are working well unless you read the transcripts and grades from many trials. At Anthropic, we invested in tooling for viewing eval transcripts and we regularly take the time to read them. When a task fails, the transcript tells you whether the agent made a genuine mistake or whether your

graders rejected a valid solution. It also often surfaces key details about agent and eval behavior.

Failures should seem fair: it's clear what the agent got wrong and why. When scores don't climb, we need confidence that it's due to agent performance and not the eval. Reading transcripts is how you verify that your eval is measuring what actually matters, and is a critical skill for agent development.

Step 7: Monitor for capability eval saturation

An eval at 100% tracks regressions but provides no signal for improvement. **Eval saturation** occurs when an agent passes all of the solvable tasks, leaving no room for improvement. For instance, SWE-Bench Verified scores started at 30% this year, and frontier models are now nearing saturation at >80%. As evals approach saturation, progress will also slow, as only the most difficult tasks remain. This can make results deceptive, as large capability improvements appear as small increases in scores. For example, the code review startup Qodo was initially unimpressed by Opus 4.5 because their one-shot coding evals didn't capture the gains on longer, more complex tasks. In response, they developed a new agentic eval framework, providing a much clearer picture of progress.

As a rule, we do not take eval scores at face value until someone digs into the details of the eval and reads some transcripts. If grading is unfair, tasks are ambiguous, valid solutions are penalized, or the harness constrains the model, the eval should be revised.

Step 8: Keep evaluation suites healthy long-term through open contribution and maintenance

An eval suite is a living artifact that needs ongoing attention and clear ownership to remain useful.

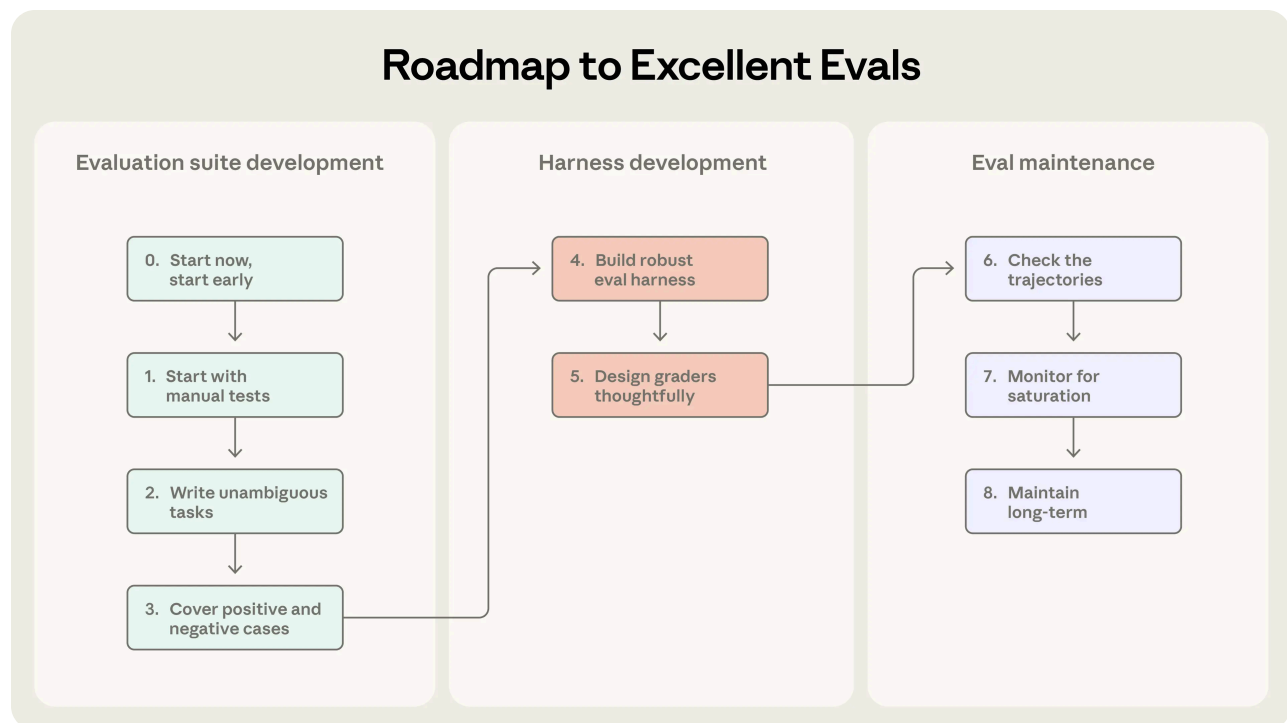
At Anthropic, we experimented with various approaches to eval maintenance. What proved most effective was establishing dedicated evals teams to own the core infrastructure, while domain experts and product teams contribute most eval tasks and run the evaluations themselves.

For AI product teams, owning and iterating on evaluations should be as routine as maintaining unit tests. Teams can waste weeks on AI features that “work” in early testing but fail to meet unstated expectations that a well-designed eval would have

surfaced early. Defining eval tasks is one of the best ways to stress-test whether the product requirements are concrete enough to start building.

We recommend practicing eval-driven development: build evals to define planned capabilities before agents can fulfill them, then iterate until the agent performs well. Internally, we often build features that work “well enough” today but are bets on what models can do in a few months. Capability evals that start at a low pass rate make this visible. When a new model drops, running the suite quickly reveals which bets paid off.

The people closest to product requirements and users are best positioned to define success. With current model capabilities, product managers, customer success managers, or salespeople can use Claude Code to contribute an eval task as a PR—let them! Or, even better, actively enable them.



The process of creating an effective evaluation.

How evals fit with other methods for a holistic understanding of agents

Automated evaluations can be run against an agent in thousands of tasks without deploying to production or affecting real users. But this is just one of many ways to understand agent performance. A complete picture includes production monitoring, user feedback, A/B testing, manual transcript review, and systematic human evaluation.

An overview of approaches for understanding AI agent performance

Method

Pros

Cons

Automated evals

Running tests programmatically without real users

- Faster iteration
 - Fully reproducible
 - No user impact
 - Can run on every commit
 - Tests scenarios at scale without requiring a prod deployment
-
- Requires more up-front investment to build
 - Requires ongoing maintenance as product and model evolves to avoid drift
 - Can create false confidence if it doesn't match real usage patterns
-

Production monitoring

Tracking metrics and errors in live systems

- Reveals real user behavior at scale
 - Catches issues that synthetic evals miss
 - Provides ground truth on how agents actually perform
-
- Reactive; problems reach users before you know about them
 - Signals can be noisy
 - Requires investment in instrumentation

- Lacks ground truth for grading
-

A/B testing

Comparing variants with real user traffic

- Measures actual user outcomes (retention, task completion)
 - Controls for confounds
 - Scalable and systematic

 - Slow; days or weeks to reach significance and requires sufficient traffic
 - Only tests changes you deploy
 - Less signal on the underlying “why” for changes in metrics without being able to thoroughly review the transcripts
-

User feedback

Explicit signals like thumbs-down or bug reports

- Surfaces problems you didn't anticipate
 - Comes with real examples from actual human users
 - The feedback often correlates with product goals

 - Sparse and self-selected
 - Skews toward severe issues
 - Users rarely explain *why* something failed
 - Not automated
 - Relying primarily on users to catch issues can have negative user impact
-

Manual transcript review

Humans reading through agent conversations

- Builds intuition for failure modes
- Catches subtle quality issues automated checks miss
- Helps calibrate what “good” looks like and grasp details

- Time-intensive
- Doesn't scale
- Coverage is inconsistent
- Reviewer fatigue or different reviewers can affect the signal quality

- Typically only gives qualitative signal rather than clear quantitative grading

Systematic human studies

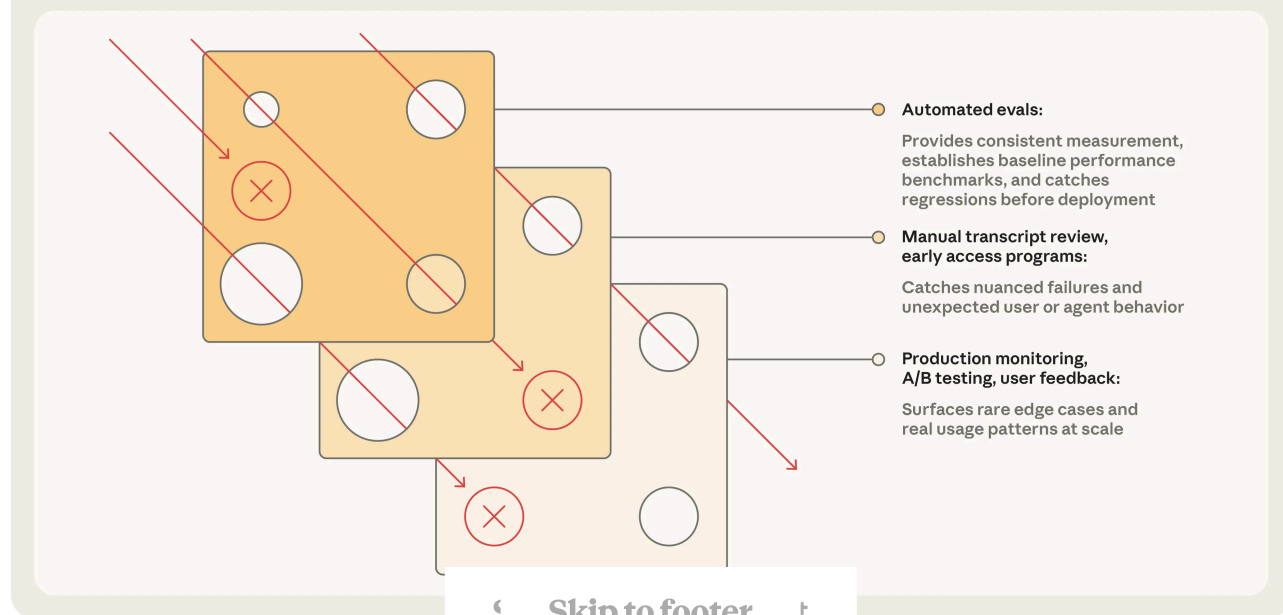
Structured grading of agent outputs by trained raters

- Gold-standard quality judgements from multiple human raters
- Handles subjective or ambiguous tasks
- Provides signal for improving model-based graders

- Relatively expensive and slow turnaround
- Hard to run frequently
- Inter-rater disagreement requires reconciliation
- Complex domains (legal, finance, healthcare) require human experts to conduct studies

These methods map to different stages of agent development. Automated evals are especially useful pre-launch and in CI/CD, running on each agent change and model upgrade as the first line of defense against quality problems. Production monitoring kicks in post-launch to detect distribution drift and unanticipated real-world failures. A/B testing validates significant changes once you have sufficient traffic. User feedback and transcript review are ongoing practices to fill the gaps: triage feedback constantly, sample transcripts to read weekly, and dig deeper as needed. Reserve systematic human studies for calibrating LLM graders or evaluating subjective outputs where human consensus serves as the reference standard.

Zooming Out: The Swiss Cheese of Quality



Like the Swiss Cheese Model from safety engineering, no single evaluation layer catches every issue. With multiple methods combined, failures that slip through one layer are caught by another.

The most effective teams combine these methods: automated evals for fast iteration, production monitoring for ground truth, and periodic human review for calibration.

Conclusion

Teams without evals get bogged down in reactive loops—fixing one failure, creating another, unable to distinguish real regressions from noise. Teams that invest early find the opposite: development accelerates as failures become test cases, test cases prevent regressions, and metrics replace guesswork. Evals give the whole team a clear hill to climb, turning “the agent feels worse” into something actionable. The value compounds, but only if you treat evals as a core component, not an afterthought.

The patterns vary by agent type, but the fundamentals described here are constant. Start early and don’t wait for the perfect suite. Source realistic tasks from the failures you see. Define unambiguous, robust success criteria. Design graders thoughtfully and combine multiple types. Make sure the problems are hard enough for the model. Iterate on the evaluations to improve their signal-to-noise ratio. Read the transcripts!

AI agent evaluation is still a nascent, fast-evolving field. As agents take on longer tasks, collaborate in multi-agent systems, and handle increasingly subjective work, we will need to adapt our techniques. We'll keep sharing best practices as we learn more.

Acknowledgements

Written by Mikaela Grace, Jeremy Hadfield, Rodrigo Olivares, and Jiri De Jonghe. We're also grateful to David Hershey, Gian Segato, Mike Merrill, Alex Shaw, Nicholas Carlini, Ethan Dixon, Pedram Navid, Jake Eaton, Alyssa Baum, Lina Tawfik, Karen Zhou, Alexander Bricken, Sam Kennedy, Robert Ying, and others for their contributions. Special thanks to the customers and partners we have learned from through collaborating on evals, including iGent, Cognition, Bolt, Sierra, Vals.ai, Macroscopic, PromptLayer, Stripe, Shopify, the Terminal Bench team, and more. This

AI

=

Appendix: Eval frameworks

Several open-source and commercial frameworks can help teams implement agent evaluations without building infrastructure from scratch. The right choice depends on your agent type, existing stack, and whether you need offline evaluation, production observability, or both.

Harbor is designed for running agents in containerized environments, with infrastructure for running trials at scale across cloud providers and a standardized format for defining tasks and graders. Popular benchmarks like Terminal-Bench 2.0 ship through the Harbor registry, making it easy to run established benchmarks along with custom eval suites.

Promptfoo is a lightweight, flexible, and open-source framework that focuses on declarative YAML configuration for prompt testing, with assertion types ranging from string matching to LLM-as-judge rubrics. We use a version of Promptfoo for many of our product evals.

Braintrust is a platform that combines offline evaluation with production observability and experiment tracking—useful for teams that need to both iterate during development and monitor quality in production. Its `autoevals` library

includes pre-built scorers for factuality, relevance, and other common dimensions.

LangSmith offers tracing, offline and online evaluations, and dataset management with tight integration into the LangChain ecosystem. Langfuse provides similar capabilities as a self-hosted open-source alternative for teams with data residency requirements.

Many teams combine multiple tools, roll their own eval framework, or just use simple evaluation scripts as a starting point. We find that while frameworks can be a valuable way to accelerate progress and standardize, they're only as good as the eval tasks you run through them. It's often best to quickly pick a framework that fits your workflow, then invest your energy in the evals themselves by iterating on high-quality test cases and graders.

Get the developer newsletter

Product updates, how-tos, community spotlights, and more.

Delivered monthly to your inbox.

Please provide your email address if you'd like to receive our monthly developer newsletter. You can unsubscribe at any time.



Products

Claude

Claude Code

Claude Code Enterprise

Cowork

Claude in Chrome